

Première Services — Comprehensive Factual Platform Audit

Technical Architecture, User Flows, Marketplace Mechanics, Data Inventory & Regulatory Touchpoints

SUBJECT ENTITY

Première Services
(premiereservices.ca)

DOCUMENT PURPOSE

Save legal fees during ToS / Privacy /
Regulatory review

AUDIT DATE

September 2026 (Codebase & Live
Schema)

REVIEWING COUNSEL

Quebec Bar / Commercial &
Regulatory Practice

Première Services — Lawyer-Ready Factual Platform Description

Document purpose: Save counsel time by describing how the website/application **currently works**, based on inspection of the repository and live Supabase schema.

Not legal advice. No assumptions presented as facts. Gaps are listed in Section 20.

Primary technical sources (non-exhaustive):

src/ , supabase/migrations/ , supabase/functions/ , src/config/legalConfig.ts , src/content/termsContent.ts ,
src/content/privacyContent.ts , src/content/cookieContent.ts , src/data/services.ts , docs/PAYMENT_FLOW.md ,
live Postgres public schema (queried via project tools).

As-of: Inspection performed September 2026 against the Première Services codebase and linked Supabase project.

1. Executive Summary

Première Services is a bilingual (English/French) Canadian home-services marketplace web application. Stack observed in code:

Layer	Technology (as used)
Frontend	React + Vite (<code>src/</code> , <code>vercel.json</code>)
Auth / DB / Storage / Edge	Supabase (<code>@supabase/supabase-js</code> , <code>supabase/</code>)
Hosting	Vercel patterns (<code>vercel.json</code> ; production domain previously attached to Vercel project <code>premier-servic</code>)
Service payments	Square Web Payments SDK + Edge Functions (<code>square-create-payment</code> , <code>square-finalize-payment</code> , Connect OAuth)
Pro plan payments	Separate Square checkout on platform credentials (<code>pro-plan-checkout</code> , <code>trial-checkout</code>)
Email	Resend API from several Edge Functions
SMS	Twilio Messages API via <code>booking-sms-notify</code> (optional if secrets present)
Maps / address	Google Places (client), Google Maps / geocoder.ca / Photon / Zippopotam (Edge <code>geocode</code>)
AI	Google Gemini + Hugging Face fallbacks (<code>ai-chat-hf</code> , <code>search-suggestions</code>)

Core product flows:

1. **Clients** register, search services by category/postal, view verified pros, post job requests and receive quotes, and/or book a pro directly on a profile page with Square payment (often authorize-then-capture).
2. **Professionals (“Pros”)** create a business profile, upload documents, await admin verification (`is_verified`), subscribe to paid plans (Starter / Growth / Pro), connect Square, receive bookings/quotes, accept/decline/complete bookings.
3. **Platform admins** approve/decline pros, view claims/job requests/trials, and (super admin) manage staff admins.

Payment characterization (technical only): Code and docs instruct **not** to call the model “escrow.” Funds move through Square (seller Connect account or platform merchant). See Section 6.

2. User Roles

Role	How identified in code	Typical capabilities
Unauthenticated visitor	No Supabase session	Browse services, categories, public pro pages (verified pros), legal pages
Client (authenticated user)	<code>auth.users</code> + <code>profiles</code> row; may have no <code>pro_profiles</code>	Book, pay, post jobs, favorites, reviews, claims, account settings
Pro (professional)	Row in <code>pro_profiles</code> for <code>user_id</code> ; public listing requires <code>is_verified = true</code>	Profile, services, bookings, quotes, Square, subscription tier features
Platform admin / moderator	Email on allowlist (<code>admin1@...</code> – <code>admin5@...</code> , <code>murc137490@gmail.com</code>) and/or <code>profiles.is_platform_admin</code> ; helpers <code>auth_is_platform_moderator()</code> , <code>auth_is_super_admin()</code>	Admin dashboard tools, approve pros, claims, job request moderation
Super admin	Email equals <code>murc137490@gmail.com</code> (<code>auth_is_super_admin()</code>); <code>SUPER_ADMIN_EMAIL</code> in <code>src/lib/platformAdminAllowlist.ts</code>	Create/revoke staff admins (<code>manage-platform-admins</code> , <code>platform_admin_staff</code>)

Same person can be both client and pro (same auth user; separate `pro_profiles` row).

3. Client Functionality

3.1 Account create / login / logout / reset

Action	Behavior	Sources
Sign up (email)	Requires full name, email, 10-digit Canadian phone , password, preferred email language EN/FR; optional referral code. Calls <code>supabase.auth.signUp</code> with metadata. Phone written to <code>profiles</code> after signup if user id exists. Links to <code>/terms</code> and <code>/privacy</code> (no <code>legal_document_acceptances</code> insert on signup found).	<code>src/pages/Auth.tsx</code> , <code>src/contexts/AuthContext.tsx</code>
Sign up / login (Google)	<code>signInWithOAuth({ provider: "google" })</code> → <code>/auth/callback</code> PKCE exchange	<code>AuthContext.tsx</code> , <code>AuthCallback.tsx</code>
Login (email/password)	Email or name → password; staff admins (not super admin) must enter 6-digit Member ID matching <code>profiles.public_user_number</code>	<code>Auth.tsx</code> , <code>adminMemberGate.ts</code>
Logout	<code>supabase.auth.signOut()</code> ; clears admin member-ID session flag	<code>AuthContext.tsx</code> , <code>Layout.tsx</code>
Email verification	Supabase Auth confirmation; templates exist under <code>supabase/email-templates/</code> and <code>send-app-email</code> types <code>auth_*</code> . No in-app invoke of <code>send-app-email</code> found under <code>src/</code> . Actual Auth email delivery depends on Supabase Auth / Resend project configuration (see Section 20).	Templates + docs
Phone verification (OTP)	Edge <code>twilio-verify</code> exists. No frontend caller found under <code>src/</code> .	<code>supabase/functions/twilio-verify/</code>
Password reset	<code>supabase.auth.resetPasswordForEmail</code> → <code>/reset-password</code>	<code>Auth.tsx</code> , <code>ResetPassword.tsx</code>

3.2 Profile edit

Dashboard **My account** (`Dashboard.tsx` tab `account`): editable `full_name` , `phone` , `postal_code` , `address` (Google Places when API key set), `email_language` , `birthday` (once set, date input disabled). Email read-only from Auth. Avatar via `uploadProfileAvatar` → storage `pro-photos` . Displays `public_user_number` (Member ID).

Admin accounts (platform admin shell): phone / postal / address / deletion request / booking ID verification UI hidden; name, email, language, birthday, Member ID remain.

3.3 Account deletion

User may insert into `account_deletion_requests` (`status: pending` , `retain_financial: true` , `retain_audit: true`). **No automated deletion of Auth user, bookings, or payments found.** Copy marks retention as `LEGAL_REVIEW_REQUIRED`. Config knob `accountDeletionGraceDays: 30` in `legalConfig.ts` is **not** identified as enforced deletion logic.

3.4 Search / browse / select pro

- Categories/services from `src/data/services.ts` .
- Postal geocode (`geocode` Edge + client helpers).
- Pro list/search filters **verified** pros (`is_verified`).
- AI-assisted search suggestions: `search-suggestions` (Gemini → Hugging Face → lexical).
- Favorites: `client_saved_pros` .

3.5 Job request → quotes

1. Client submits `job_requests` (`MakeRequest.tsx`) with description, category, location, photos (`job-request-photos` bucket), scheduling fields.
2. Status default `open` .
3. Pros insert `job_quotes` (price, message, optional proposed date).
4. Client accepts quote: **must pay via Square (capture)** then `job_quotes.status = accepted` .
5. Client may decline quote → `declined` .
6. **Accept handler does not insert a `bookings` row** (fact from `Dashboard.tsx` quote accept flow).
7. Job requests older than 7 days can be purged (`purge_job_requests_older_than_seven_days` / `purgeStaleJobRequests.ts`).

3.6 Direct book on pro profile

On `/pros/:proId` (`ProProfilePage.tsx`):

1. Select service / schedule / location.
2. Accept **client booking terms** (`TermsAcceptance` → `legal_document_acceptances` with `client_booking_terms`).
3. Acknowledge cancel policy (checkbox; snapshots stored on booking).
4. Phone required.
5. Optional client booking ID verification upload (`client-booking-verification` bucket).

- Square payment with `authorizeOnly` (card authorization, not immediate capture) via `BookingRequestConfirm` / `SquareBookingPayment` → `square-create-payment` .
- Insert `bookings` with `status: pending` + invoice/cancel snapshots.
- SMS confirmation may fire via `booking-sms-notify` if Twilio configured.

3.7 Pay / cancel / refund / claim / review / photos / notifications

Topic	Client-side fact
Pay after accept	If accepted without payment row: "Pay now" dialogs → capture
Cancel booking in app	Cancel policy disclosed/acked; no client UI found that sets <code>bookings.status</code> to <code>cancelled</code> .
Refund	Via claim workflow / admin resolution string; no automated Square Refunds API found
Report / claim	<code>BookingClaimDialog</code> → <code>booking_claim_requests</code> + evidence in private <code>booking-evidence</code> ; emails via <code>send-booking-claim-email</code>
Communicate with Pro	No general in-app messaging table found. Contact via booking/phone/email fields and support AI route
Review	After booking relationship: <code>reviews</code> (+ photos). Blind until mutual (<code>reviewBlind.ts</code>). One per pair
Notifications	In-app unread flags on bookings; Whats New announcements; emails (claim/decline/referral); SMS booking notify

4. Pro Functionality

4.1 Create business profile

`ProProfileEditorDialog` / `join-pros` / `create-pro-account` flows:

- Business name, bio, location, phone, website, years experience, prices, services (`pro_services`), photos (`pro-photos`), tags, aesthetic, availability calendars, cancel policies, workspace/travel modes, billing fields (`legal_business_name` , `business_address` , GST/QST numbers).
- Required on create:** personal photo + ID document stored under `pro-photos` private paths (`personal_photo_url` , `id_document_url`).

- Accepts **professional agreement** via `TermsAcceptance` → `legal_document_acceptances` .
- Starts `is_verified = false` until admin approval.

4.2 Verification / licences

Item	Fact
Admin sets <code>is_verified = true</code>	<code>accept-pro</code> Edge Function / Dashboard admin
Public listing	Search/list requires verified
Badge	“Verified” UI + disclaimer in <code>legalConfig</code> / i18n (not warranty)
<code>pro_licenses</code> table	Exists; displayed on credentials tab. No insert UI found in pro editor in this audit
Insurance certificates	Not identified as a dedicated upload/field in this audit beyond general docs

4.3 Services / pricing / quotes / bookings

- CRUD-ish services in Dashboard (tier-gated features via subscription).
- Receive open job requests; submit quotes.
- Incoming bookings: see client `full_name` , `phone` , booking ID verification **status** (and historically storage path access—see security section).
- Accept → capture Square auth; Decline → void + `send-booking-declined-email` .
- Mark **completed**.
- Pro reviews of clients: `client_reviews` .
- Pro reply to client reviews: `review_responses` .

4.4 Payments / payouts / plans

- Connect Square OAuth → `pro_square_tokens` + `square_location_id` .
- Service charges on seller account when Connect active (+ `app_fee_money`).
- Subscriptions: Starter / Growth / Pro via `pro-plan-checkout` ; trials via `trial-checkout` ; cancellations recorded in `pro_plan_cancellations` .
- **Payout timing to Pro bank account:** Not controlled in Première code; Square seller settlement. Not determinable from codebase beyond that.

4.5 What Pro sees about Client vs Client about Pro

Pro can see about client (from booking/dashboard code):

`full_name` , `phone` , booking schedule/service, invoice-related booking fields, booking ID verification status (and photo path fields selected in queries). Address/postal may appear on job requests the pro quotes.

Client can see about pro (public profile):

Business name, bio, location/radius modes, services/prices, photos, reviews (subject to blind rules), verified badge, licenses if present, contact fields as displayed (public phone/website may be restricted by blacklist helpers).

Client generally cannot see: Pro private ID/selfie documents (admin/moderator storage policies), Square tokens, other clients' PII.

5. Booking Lifecycle

5.1 Actual `bookings.status` values (DB)

From migration `20260511120000_booking_growth_bundles_notifications.sql` and app usage:

`pending` | `accepted` | `completed` | `cancelled` | `declined`

5.2 Implemented transitions

Transition	Who	Conditions / side effects	Sources
→ <code>pending</code>	Client	After Square authorization ; booking insert	<code>ProProfilePage.tsx</code> , <code>BookingRequestConfirm.tsx</code>
<code>pending</code> → <code>accepted</code>	Pro	Capture via <code>square-finalize-payment</code> <code>complete</code>	<code>Dashboard.tsx</code> <code>handleApproveBooking</code>
<code>pending</code> → <code>declined</code>	Pro	Void via <code>cancel</code> ; decline email	<code>handleDeclineBooking</code>
<code>accepted</code> → <code>completed</code>	Pro	Mark complete	<code>handleMarkBookingComplete</code>
→ <code>cancelled</code>	DB allows	No production UI write to <code>cancelled</code> found	—

Not found as booking statuses: REQUESTED, QUOTE, PAYMENT, CONFIRMED, SCHEDULED, PROVIDER_ARRIVED, SERVICE_IN_PROGRESS, DISPUTE_WINDOW, PAYOUT_RELEASED.

Quote lifecycle is separate (`job_quotes` : `pending` / `accepted` / `declined`).

5.3 Scenario matrix (software behavior)

Scenario	What code does
Client cancel	Policy disclosed; no automated status/fee charge path found
Pro cancel	Decline path voids auth; decline email
Pro no-show	No dedicated status/automation found
Incomplete / poor service	Client claim categories → admin resolution
Dispute	<code>booking_claim_requests</code> + emails; admin sets <code>admin_resolution</code>
Payment fail	Square error surfaced; booking may not complete authorize path
Refund	Admin marks resolution; no Square refund API call found
Auth void	Decline or finalize <code>cancel</code> on uncaptured payment

6. Payment Architecture

6.1 Processors

- **Sole processor: Square.** Live booking, quotes, and subscriptions run exclusively through Square (Connect + authorize/capture).
- **Stripe removed:** Legacy orphan Stripe packages (`@stripe/stripe-js` , `@stripe/react-stripe-js`), component (`StripeBookingPayment.tsx`), and Edge function (`create-payment-intent`) were completely removed from the codebase in September 2026 to ensure zero dual-processor ambiguity.

6.2 Booking charge mechanics

1. Client tokenizes card/wallet in browser (Square Web Payments SDK).
2. Edge `square-create-payment` creates payment.

3. If pro has Connect tokens + location: charge on **pro seller**; `app_fee_money` $\approx$ **2.1%** of service subtotal (`SQUARE_CONNECT_APP_FEE_RATE`).
4. Else: charge on **platform** Square merchant (no `app_fee_money`).
5. Profile booking uses **authorize-only** then capture/void on accept/decline.
6. Other paths capture immediately (pay-now, quote accept, plans).

6.3 Amounts (customer-facing invoice math)

Implemented in `src/lib/bookingInvoiceAmounts.ts` + `legalConfig.ts` :

- Service subtotal
- GST **5%** on subtotal
- QST **9.975%** on (subtotal + GST)
- Platform/processing line **5%** of service subtotal (`PLATFORM_FEE_RATE`)
- Total charged to client

Internal Connect application fee default **2.1%** of service subtotal is separate from the 5% line shown to users.

6.4 Stored payment data

Table `payments` : `square_payment_id` , `idempotency_key` , `amount_cents` , `currency` (CAD), `status` , `booking_id` , `pro_profile_id` , `client_id` , `card_brand` , `card_last_4` , `created_at` .

Also: `bookings.invoice_snapshot` (jsonb), `apple_pay_handoffs` , `pro_square_tokens` (access/refresh tokens—server-side).

Full PAN / CVV: Not stored by Première app code; handled by Square.

6.5 Escrow

Docs and Terms explicitly state the platform does **not** operate traditional escrow merely by using Square. Authorize/capture is Square authorization, not an escrow product in code.

6.6 Subscriptions

Plans `starter` / `growth` / `pro` (and `hold`). Prices in checkout Edge Function (defaults documented as CAD cents 2000 / 2700 / 3200). Stored in `pro_subscriptions` / `subscription_plans` . Cancellation creates `pro_plan_cancellations` rows.

7. Refunds / Cancellations / Disputes

Mechanism	Implementation		
Cancel policies	<code>free</code> / <code>late_fee</code> (<24h fee 25/50/75%) / <code>no_cancel</code> ; snapshotted on booking; UI acknowledgment		
Automated cancel fee charge	Not found		
Claims	Categories → <code>claim_type</code> + <code>dispute_category</code> ; evidence private bucket; Resend emails		
Admin resolution	<code>refunded</code> \	<code>job_redone</code> \	<code>resolved</code> (+ workflow fields); manual relative to Square
Chargebacks	Not implemented in app code ; would be Square/processor side		
Payout already released	Not modeled in app; Square settlement timing		

8–10. Data Collection, Storage, Access

8.1 Inventory (selected categories)

Data	Purpose (from product use)	Provider	Access	Storage
Email, password hash	Auth	User / Auth	Self; admins via tools	Supabase Auth
<code>full_name</code> , phone, address, postal, birthday, avatar, email_language	Profile / booking contact	User	Self; booking counterparty/admin as coded	<code>profiles</code>
<code>public_user_number</code>	Public member ID	System	Visible on account	<code>profiles</code>
Client ID verification image	Booking ID check	Client	Self; policies historically allowed booking pros to read	<code>client-booking-verification</code> + path on profile
Pro business profile + private ID/selfie	Application / verification	Pro	Self; moderators	<code>pro_profiles</code> + <code>pro-photos</code>
Bookings, invoices snapshots	Marketplace ops	Client/Pro	Parties + admins	<code>bookings</code>
Payments metadata	Payment audit	Square + system	Pro/client/admin as RLS	<code>payments</code>
Square tokens	Connect charging	OAuth	Service role / Edge	<code>pro_square_tokens</code>
Job requests/quotes/photos	Lead marketplace	Client/Pro	As RLS	tables + <code>job-request-photos</code>
Reviews / photos / responses	Reputation	Users	Public select; writers own	<code>reviews*</code> / <code>client_reviews</code>
Claims + evidence	Disputes	Client	Client/admin; evidence private	<code>booking_claim_requests</code> , <code>booking-evidence</code>

Data	Purpose (from product use)	Provider	Access	Storage
Legal acceptances	Proof of assent	User	Self	<code>legal_document_acceptances</code>
Cookie preferences	Consent	User	Browser	<code>localStorage</code> (<code>cookieConsent.ts</code>)
Account deletion request	Offboarding request	User	Self	<code>account_deletion_requests</code>
Admin staff HR fields	Staff management	Super admin	Super admin	<code>platform_admin_staff</code>
Audit events	Admin actions	System	Super admin	<code>platform_admin_audit_events</code>

Retention/deletion: Automated job-request purge after 7 days. Account deletion is a **request row** with financial/audit retain flags default true. ID verification / claim evidence retention days in `legalConfig` are `null`. Otherwise: **Not identified in the codebase.**

Cross-border: Supabase/Vercel/Square/Resend/Google/Twilio are third-party processors; **exact physical region of production data not determinable from app source alone** (see Section 20).

10.1 RLS highlights

- Many tables: own-row policies; bookings SELECT for client OR owning pro OR platform admin.
- Public SELECT on pro profiles and reviews.
- Some sensitive tables have RLS enabled with **no** authenticated policies (service-role only), e.g. `pro_square_tokens`, `privacy_security_incidents`, `platform_admin_config` (as reported from live policies).
- `profiles` SELECT policy includes broad “Anyone can view profiles” (`qual true`) — personal fields may be readable more widely than product UI suggests; **counsel should review.**

11. Third-Party Services

Service	Status	What is sent (high level)
Supabase	Used	Auth, DB, Storage, Edge runtime
Vercel	Used (hosting)	Static/app assets, build env
Square	Used	Payment tokens, charges, OAuth, Apple Pay domain
Resend	Used	Transactional emails (claims, declines, referrals, admin emails, <code>send-app-email</code> capability)
Google OAuth	Used	Identity for login
Google Places/Maps	Used when keys set	Address autocomplete / geocode
Gemini / Hugging Face	Used in Edge AI/search	Prompt/search text (no guarantee of PII filtering beyond app prompts)
Twilio SMS	Used if secrets set	Booking SMS to phone numbers
Twilio Verify	Present in Edge; unused by frontend found	
Stripe	Removed	Previously had unused prototype files; packages and code completely removed in September 2026
geocoder.ca / Photon / Zippopotam	Used by <code>geocode</code> Edge	Postal/location queries
Analytics SDKs (GA, etc.)	Not found in <code>package.json</code> / <code>src</code>	Cookie UI has analytics category only

12. Privacy / Security Implementation (descriptive)

- Auth: Supabase email/password + Google OAuth; JWT to Edge Functions.
- Authorization: RLS + `auth_is_platform_moderator` / `auth_is_super_admin` + client allowlists.
- Admin Member ID gate for staff (not super admin).
- Storage: mix of public (`pro-photos` , `review-photos` , `job-request-photos` , `pro-public`) and private (`booking-evidence` , `client-booking-verification`) buckets.

- Signed URL TTL config: `signedUrlTtlSeconds: 300` in `legalConfig`.
- Password reset via Supabase.
- `privacy_security_incidents` table exists; **no authenticated write UI found**.
- Backups / encryption-at-rest: **provider-managed; not described in app code**.
- Error logging: client diagnostics silencing helpers; Edge logs via Supabase—**not a full SIEM**.

This section describes mechanisms present; it does **not** assert overall “security adequacy.”

13. Reviews

Rule	Fact
Who	Client reviews pro (<code>reviews</code>); pro reviews client (<code>client_reviews</code>)
When	After booking relationship; UI gates
Edit	RLS allows; no edit UI found ; copy says definitive
Delete	RLS allows; locks prevent re-submit; no delete UI call found under <code>src/</code>
Pro response	<code>review_responses</code>
Blind	Content blurred until both sides posted
Moderation	No review moderation admin tool found
Photos	Supported
Public	SELECT policies allow public read of reviews
Ranking	Ratings used in display/rank helpers (<code>get_pro_avg_rating</code> etc.)

14. Verification / Licensing

Item	Technically enforced	Merely displayed / claimed
Admin <code>is_verified</code>	Yes — listing/search filters	Badge with disclaimer
Pro ID + selfie upload	Required in create UI	Admin visually reviews
RBQ / licence numbers	Table exists	Display if present; no automated regulator API found
Client ID for booking	Upload + status fields	Pro sees status; not a government e-KYC integration found
Insurance	—	Not found as structured enforced field

15. Communications

Channel	Trigger examples	Provider
Resend email	Claim submitted; booking declined; referral invite; decline/remove pro; <code>send-app-email</code> template types (booking_, support_, auth_*)	Resend
SMS	Booking confirmation/reminder from profile book	Twilio (if configured)
In-app	Booking unread flags; Whats New; toasts	App / Supabase
Cookie banner	First visit preferences	localStorage
Marketing email/SMS campaigns	No dedicated marketing campaign system found	—

16. Administrator Functionality

Observed admin capabilities:

- Approve / decline / remove pros; review ID/selfie and application diffs
- View subscriptions/payment enrollment when reviewing applicants

- Admin job requests moderation (strikes/notices)
- Issue reports / claims review and resolution fields
- Trial token management
- Whats New announcements
- Platform admin shell dashboard (accept pros, shortcuts)
- Super admin: create/revoke staff with Member IDs, HR fields, audit events
- Staff admin login requires Member ID; super admin exempt

Not found as automated admin tools: issuing Square refunds, editing arbitrary bookings' financial capture, full user soft-delete execution beyond deletion **requests**.

17. Complete Service Inventory

Source: `src/data/services.ts` (catalog). Hierarchy abbreviated here; full list matches audit extraction:

- **Home Improvement** — remodel, plumbing, electrical, HVAC, roofing, windows/doors, flooring, painting, appliances
- **Outdoor & Seasonal** — snow, lawn, landscaping, trees, driveway, deck/patio
- **Cleaning** — house, deep, move-in/out, carpet, window, commercial, pressure washing
- **Business Services** — accountant, tax, bookkeeping, web, SEO, marketing, IT
- **Events & Entertainment** — photo/video, DJ, catering, planning, entertainer
- **Lessons & Tutoring** — math, French, English, music, driving
- **Pets** — walking, sitting, grooming, training
- **Wellness** — trainer, massage, counselling, nutritionist
- **Moving & Storage** — local/long-distance moving, furniture assembly, TV mount, appliance install
- **Home Security & Inspection** — home inspection, pest, locksmith, security systems, property management

No migration seeds the full catalog into a services table for production listing; pros attach catalog slugs via `pro_services`.

18. Existing Legal Documents

Document	Route / placement	EN/FR	Affirmative acceptance recorded?
Website Terms of Service	<code>/terms</code> (<code>TermsOfService.tsx</code> + <code>termsContent.ts</code>)	Yes	No <code>legal_document_acceptances</code> for <code>website_terms</code> found
Provider / professional agreement text	Sections on <code>/terms</code> ; summary in pro editor	Yes	Yes — <code>professional_agreement</code> via <code>TermsAcceptance</code>
Client booking terms	Summary in booking UI; link to <code>/terms</code>	Yes	Yes — <code>client_booking_terms</code>
Privacy Policy	<code>/privacy</code> (+ redirects)	Yes	No recorded acceptance found
Cookie Policy	<code>/cookies</code> (+ redirects)	Yes	Preferences in localStorage , not <code>legal_acceptances</code> table
Cancellation policy framework	Config key + booking cancel checkbox	Partial	Cancel ack timestamp on booking; framework key not logged as <code>legal_acceptances</code>

Versions/hashes: draft strings in `legalConfig.ts` (`2026-03-draft` , `2026-08-draft`). Pages may also show content-level “last updated” constants.

19. Facts Requiring Legal Review

(Factual features for counsel to examine — not conclusions of legality.)

1. **Marketplace vs merchant model** — Connect seller charges vs platform merchant charges coexist.
2. **Authorize/capture timing** vs customer expectations of when money is taken.
3. **5% customer fee line** vs **2.1% app_fee_money** vs Square processing costs.
4. **GST/QST computation** on invoices and who remits tax.
5. **Absence of automated refunds** while UI/admin can mark “refunded.”
6. **Cancel policies disclosed but not auto-enforced** in status/fee charging.
7. **Job quote accept creates payment without** `bookings` **row**.
8. **Account deletion is request-only** with default retain financial/audit.

9. **Broad profiles SELECT RLS** (`true`) vs privacy expectations.
10. **Client ID images** and pro access policies historically/currently.
11. **Pro ID/selfie storage** and admin access.
12. **Cross-border processors** (US-based vendors likely).
13. **Google/Gemini/HF** processing of user search/chat text.
14. **SMS** without Twilio Verify wired to signup.
15. **Cookie analytics category** without analytics SDK.
16. **French-language** presentation and contract language.
17. **Age** — birthday min-age helpers exist; signup does not clearly collect DOB at Auth.
18. **Verified badge disclaimer** vs consumer reliance.
19. **Reviews** public + blind mutual + limited moderation tooling.
20. **Electronic assent** only for booking/pro summaries, not full website terms/privacy.
21. **Admin power** over personal data and approvals.
22. **Regulated professions** listed (electrical, plumbing, counselling, etc.) without automated licence validation.
23. **“Not escrow”** statements vs authorize-hold UX.
24. **Apple Pay / Google Pay** wallet flows and handoff QR.
25. **Referral/trial** incentives and payment method collection on trials.

20. Information Not Determinable From the Codebase

- Legal entity name, directors, registered address, Quebec enterprise number
- Insurance coverage (platform or pros) actually purchased
- Contracts between Première and Square/Resend/Twilio/Google/Vercel/Supabase (commercial terms)
- Actual production Supabase/Vercel **region** and subprocessors list in force
- Whether Auth confirmation emails are currently delivered via Resend vs Supabase SMTP in production
- Manual back-office procedures (how admins decide refunds, how long they keep evidence)
- Tax registration and remittance practices
- Whether DNS/CDN (Cloudflare observed on responses) stores logs and where
- Intended future features not implemented

- Real-world frequency of Connect vs legacy platform charging
- Employee/contractor agreements for platform admins
- Penetration test / SOC reports
- Exact card data flows inside Square’s systems beyond API usage

Appendix A — Key file index

Topic	Paths
Roles / admin	src/lib/platformAdminAllowlist.ts , src/hooks/usePlatformAdmin.ts , supabase/migrations/20260904010000_super_admin_staff.sql
Auth	src/contexts/AuthContext.tsx , src/pages/Auth.tsx , AuthCallback.tsx
Booking UI	src/pages/ProProfilePage.tsx , Dashboard.tsx
Payments	supabase/functions/square-create-payment , square-finalize-payment , src/components/SquareBookingPayment.tsx , docs/PAYMENT_FLOW.md
Fees/tax	src/config/legalConfig.ts , src/lib/bookingInvoiceAmounts.ts
Claims	BookingClaimDialog.tsx , src/lib/disputeCategories.ts , send-booking-claim-email
Legal content	src/content/termsContent.ts , privacyContent.ts , cookieContent.ts , TermsAcceptance.tsx , legalAcceptance.ts
Services catalog	src/data/services.ts
Types (partial)	src/integrations/supabase/types.ts (incomplete vs live schema)

End of factual briefing. Counsel should verify production configuration (secrets, Auth email provider, live RLS) against this map; configuration can differ from repository defaults.